

KAIRO

Local-first desktop workspace
для проектов, задач, заметок и фокуса

Единое техническое задание, собранное на основе концепций Flowdesk и FocusHub. Цель — не просто сделать task manager, а показать полноценный уровень разработки desktop-продукта.

Параметр	Решение
Формат	Кроссплатформенное desktop-приложение
Подход	Local-first, offline-first
Основной стек	Tauri 2.x + React + TypeScript + SQLite
Целевая аудитория	Фрилансеры, разработчики, дизайнеры, студенты, small-team leads
Главная идея	Project → Task → Focus → Note → Review

Версия ТЗ: 1.0 · Рабочее название проекта: Kairo

1. Концепция продукта

Kairo — локальное desktop-приложение для управления проектами и личной продуктивностью. Оно объединяет проекты, задачи, markdown-заметки, фокус-сессии, аналитику и локальные файлы в одном рабочем пространстве.

Ключевой пользовательский цикл:

Plan → Focus → Work → Capture → Review

В отличие от обычного task manager, приложение должно показывать не только список задач, но и связь между тем, что пользователь делает, над каким проектом, какие заметки он оставляет и сколько времени реально потратил.

1.1. Ценность

- Один локальный инструмент вместо разрозненных task manager + notes + Pomodoro + time tracking.
- Работа без регистрации и обязательного сервера.
- Быстрый desktop UX: горячие клавиши, command palette, tray, быстрые действия.
- Данные принадлежат пользователю: локальное хранение и явный export/import.
- Проект демонстрирует полный цикл разработки: архитектуру, БД, UI/UX, native desktop API, аналитику и packaging.

1.2. Целевая аудитория

- Фрилансеры и индивидуальные разработчики.
- Дизайнеры и создатели цифровых продуктов.
- Студенты и люди, ведущие несколько учебных/личных проектов.
- Небольшие команды, которым нужен лёгкий локальный workspace без сложной серверной инфраструктуры.

2. Что объединено из Flowdesk и FocusHub

Ниже — не копирование одного из исходных ТЗ, а сознательное объединение их сильных сторон.

Источник	Что переносим в Kairo	Роль в итоговом продукте
Flowdesk	Projects / workspace	Проект становится верхнеуровневой сущностью
Flowdesk	Task ↔ Note связи	Контекст работы не теряется
Flowdesk	Local files	Desktop-интеграция с файловой системой
Flowdesk	Dashboard	Единая стартовая точка
Flowdesk	Command palette	Быстрые действия без мыши
Flowdesk	Архитектурное разделение	UI / domain / repositories / native layer
FocusHub	Kanban	Визуальное управление задачами
FocusHub	Pomodoro / Focus	Фокус-сессия становится частью workflow
FocusHub	Analytics	Измеримый результат работы
FocusHub	Streak	Мотивационный, но вторичный показатель
FocusHub	Tray + notifications	Нативный desktop-сценарий

Источник	Что переносим в Kairo	Роль в итоговом продукте
FocusHub	Export / import	Контроль над локальными данными
FocusHub	RU / EN + demo data	Готовность для демонстрации портфолио

Ключевое архитектурное решение: Project — отдельная сущность верхнего уровня. Board является представлением задач внутри проекта, а не заменой понятию проекта. Это делает модель данных более логичной и масштабируемой.

3. Анализ референсных паттернов

Продукты из исходного анализа	Паттерн	Как используется
Things 3 / Todoist	Быстрый ввод, типографика, hotkeys	Quick add + keyboard-first UX
Notion / Obsidian	Sidebar, Markdown, локальные данные	Notes и структура workspace
Things / Linear	Kanban и drag & drop	Boards проектов
Forest / Session	Focus timer	Фокус-сессии, уведомления
RescueTime / Toggl Track	Аналитика времени	Dashboard и отчёты
Raycast / Spotlight	Command palette	Cmd/Ctrl+K и быстрые действия
Arc / Linear	Современный desktop UI	Тема, панели, микроанимации

Вывод исходного анализа сохраняется: Kairo не должен пытаться заменить все перечисленные продукты во всех сценариях. Он берёт лучшие паттерны как основу одного цельного workflow и ограничивает MVP тем, что усиливает центральный сценарий проекта.

4. Информационная архитектура

Основная структура приложения:

```

Workspace
├── Dashboard
├── Projects
│   ├── Project
│   │   ├── Board / Tasks
│   │   ├── Notes
│   │   ├── Files
│   │   └── Focus history
│   ├── Notes
│   ├── Focus
│   ├── Analytics
│   └── Settings

```

4.1. Главный экран Dashboard

- Сегодня: задачи, активный проект, текущая/последняя focus-сессия.
- Прогресс по активным проектам.
- Краткая статистика: completed tasks, focus time, sessions.
- Быстрые действия: New task, Start focus, New note, Open project.
- Список недавно изменённых заметок/проектов.

5. Функциональные требования

5.1. Projects

- Создание, редактирование, архивирование и удаление проекта.
- Поля: название, описание, статус, цвет/иконка, дата создания.
- Внутри проекта доступны Board, Notes, Files и статистика.
- Проект можно открыть из dashboard, command palette и поиска.
- Статусы проекта: Active / Paused / Completed / Archived.

5.2. Tasks / Kanban

- Стандартные колонки: Backlog, In Progress, Done.
- Пользователь может создавать, переименовывать, удалять и переставлять колонки.
- Карточка задачи: title, description, priority, tag/color, deadline, checklist, project.
- Drag & drop между колонками и изменение порядка.
- Inline quick creation.
- Поиск и фильтры по тексту, проекту, тегам, сроку, статусу.
- Task detail открывается в side panel/modal без потери контекста.
- Задача может быть связана с заметкой и focus-сессиями.

5.3. Notes

- Markdown editor с live preview.
- Заголовки, списки, bold/italic, checkbox, ссылки и базовое форматирование.
- Папки/теги и список заметок.
- Поиск по содержимому.
- Связь Note ↔ Project и Note ↔ Task.
- Autosave и индикатор состояния сохранения.
- Локальное хранение содержимого.

5.4. Files

- Привязка локальной папки/файлов к проекту.
- Открытие файла системным приложением.
- Открытие папки в системном файловом менеджере.
- Хранение ссылок/путей, а не обязательное копирование всего содержимого внутрь БД.
- Обработка отсутствующего или перемещённого файла с понятным empty/error state.

5.5. Focus / Pomodoro

- Work / Short break / Long break.
- Настраиваемая длительность.
- Start / Pause / Resume / Stop.
- Выбор задачи перед началом сессии.
- После завершения сессия сохраняется в историю.
- System notification и звук.
- Оставшееся время доступно через system tray.
- Фокус-сессия автоматически учитывается в аналитике.

5.6. Analytics

- Статистика за день и неделю.
- Completed tasks.
- Количество focus-сессий.
- Общее focus time.
- Focus time по проектам и тегам.
- Streak как дополнительный показатель.
- Фильтрация периода.

5.7. Search + Command Palette

- Cmd/Ctrl+K открывает command palette.
- Действия: create task, create note, open project, start focus, search, settings.
- Глобальный поиск по задачам, заметкам и проектам.
- Keyboard navigation и быстрый Enter для действия.

5.8. Data management

- Полный export/import.
- JSON для переносимости.
- SQLite backup для технического резервного копирования.
- Проверка схемы перед импортом.
- Понятное предупреждение, что экспорт без дополнительного шифрования не является защищённым хранилищем.

5.9. Onboarding / Demo mode

- 3–4 коротких onboarding-экрана.
- Опциональная загрузка demo data.
- Demo workspace должен позволять за 1–2 минуты увидеть Projects → Tasks → Notes → Focus → Analytics.
- После onboarding пользователь может сразу открыть рабочий dashboard.

5.10. Settings

- Light / Dark / System theme.
- RU / EN.
- Настройки timer.
- Уведомления и звук.
- Папка/параметры локальных данных при необходимости.
- Export / Import.
- О приложении и версия.

6. UX-требования

6.1. Принципы

- Desktop-first, keyboard-friendly.
- Минимум лишних экранов и modal-overload.
- Действие должно давать визуальный feedback.
- Состояние данных всегда понятно: saved / saving / error.
- Пустые состояния не являются заглушками: они объясняют следующий шаг.
- Ошибки формулируются человеческим языком и предлагают действие.

6.2. Обязательные UI states

- Loading.
- Empty project.
- Empty board.
- No notes.
- No search results.
- No analytics yet.
- Saving / saved.
- Validation error.
- Database error.
- File not found.
- Import conflict / invalid backup.
- Timer active / paused / completed.

6.3. Keyboard shortcuts

Shortcut	Action
Cmd/Ctrl + K	Command palette
N	New task в соответствующем контексте
Cmd/Ctrl + Enter	Save / confirm
Esc	Close panel / cancel
Space	Pause/resume timer при активном timer context
Cmd/Ctrl + P	Global search / quick open

7. Дизайн-система

- Tailwind CSS + собственные React-компоненты; тяжёлый UI-kit не обязателен.
- Шрифт: Inter или Manrope; monospace — для технических значений.
- Нейтральная графитовая/светлая база + один brand accent.
- Dark theme проектируется отдельно, а не просто инвертирует цвета.
- Border radius умеренный; интерфейс должен ощущаться как desktop tool, а не web landing page.
- Микроанимации 150–250 ms.
- Drag & drop должен иметь понятное визуальное состояние.

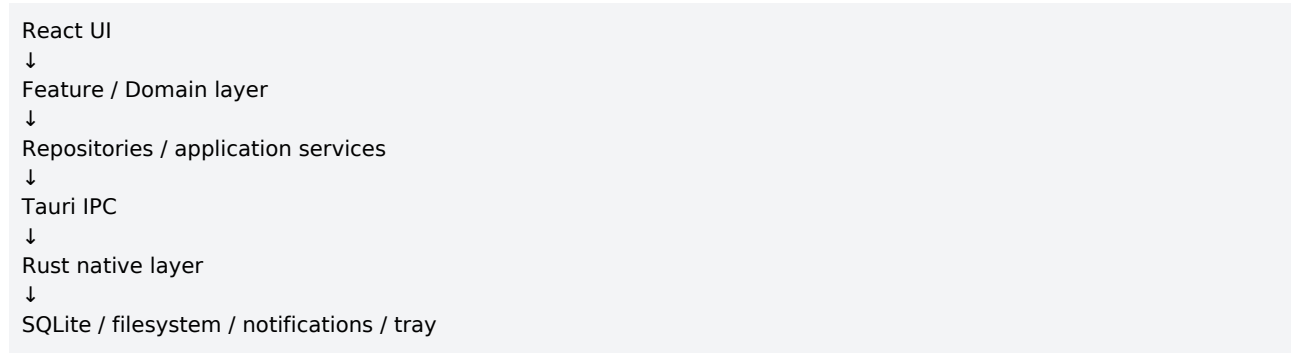
- Минимальный размер окна: 1024×640.

7.1. Ключевые экраны

Экран	Основное содержимое
Onboarding	Ценность продукта + demo data
Dashboard	Today, projects, focus, statistics
Projects	Список и статусы проектов
Project	Board + notes + files + focus/history
Task panel	Детали задачи, checklist, связи
Notes	Список + editor + preview
Focus	Timer, task, session controls
Analytics	Charts, trends, breakdowns
Settings	Appearance, language, data, timer

8. Архитектура

Рекомендуемая архитектура сохраняет сильную сторону Flowdesk: UI не должен напрямую смешивать rendering, бизнес-логику и native API.



8.1. Frontend

- React 18 + TypeScript + Vite.
- Zustand для UI/state cache.
- dnd-kit для Kanban.
- Recharts для аналитики.
- React Hook Form + Zod для сложных форм.
- Markdown renderer/editor с контролируемой безопасностью ссылок и HTML.

8.2. Desktop / native

- Tauri 2.x.
- Rust используется тонким слоем для native-команд.
- SQLite — источник истины для persistent data.
- Filesystem API для работы с локальными файлами.
- Notification API для завершения focus-сессии.
- Tray API для фонового таймера и быстрых действий.

- Bundler для Windows/macOS/Linux.

8.3. Правило источника истины

SQLite и native layer являются source of truth для persistent data. Zustand хранит состояние UI и кэш последнего полученного состояния. Это предотвращает расхождение между интерфейсом и базой данных.

9. Технологический стек

Область	Технология
Desktop shell	Tauri 2.x
Native	Rust
UI	React + TypeScript
Build	Vite
Styling	Tailwind CSS
State	Zustand
Drag & drop	dnd-kit
Charts	Recharts
Validation	Zod + React Hook Form
Database	SQLite
Testing	Vitest + Playwright
Packaging	Tauri bundler

10. Модель данных

Предлагаемая схема:

```
WORKSPACE 1—N PROJECTS
PROJECT 1—N COLUMNS
COLUMN 1—N TASKS
TASK 1—N SUBTASKS
TASK 1—N FOCUS_SESSIONS
TASK N—N TAGS
NOTE N—N TAGS
NOTE N—N TASKS
PROJECT 1—N NOTES
PROJECT 1—N FILE_REFERENCES
```

10.1. Таблицы

Table	Основные поля
workspaces	id, name, created_at
projects	id, workspace_id, name, description, status, color, created_at, updated_at
columns	id, project_id, name, position
tasks	id, project_id, column_id, title, description, priority, due_at, position, created_at, updated_at
subtasks	id, task_id, title, completed, position

Table	Основные поля
tags	id, name, color
task_tags	task_id, tag_id
notes	id, project_id, title, content, created_at, updated_at
note_tags	note_id, tag_id
note_tasks	note_id, task_id
file_references	id, project_id, path, label, created_at
focus_sessions	id, task_id, project_id, started_at, ended_at, duration_sec, type
settings	key, value

Рекомендуется хранить position как числовой порядок внутри колонок/списков и обновлять его транзакционно при reorder.

11. IPC API

Domain	Commands
Projects	get_projects, create_project, update_project, archive_project, delete_project
Tasks	get_tasks, create_task, update_task, move_task, delete_task
Notes	get_notes, save_note, delete_note, search_notes
Files	get_file_references, add_file_reference, remove_file_reference, open_file, reveal_in_folder
Timer	start_timer, pause_timer, resume_timer, stop_timer, complete_session
Analytics	get_analytics
Data	export_data, import_data, validate_backup
Settings	get_settings, update_setting

IPC-команды должны валидировать входные данные и возвращать структурированные ошибки, а не необработанные Rust exceptions.

12. Производительность и надёжность

Метрика / требование	Цель
Cold start	≤ 1–1.5 сек на референсном desktop
Отклик обычного UI-действия	< 100 ms, где операция не требует дискового запроса
Autosave	Без заметных задержек интерфейса
Offline	Основные функции доступны без сети
Данные	SQLite + транзакции для критических изменений
Crash resilience	Изменения не должны теряться из-за обычного UI-сбоя
Размер сборки	Ориентир 15–30 MB; проверяется после packaging

Числовые цели являются инженерными ориентирами из исходных ТЗ, а не гарантией. Их следует измерить на конкретной референсной машине и указать в portfolio case study фактическими значениями.

13. Безопасность и приватность

- Приложение не требует регистрации для основной работы.
- Основные данные хранятся локально.
- Пути к локальным файлам не должны отправляться в сеть без явной функции, которая это предусматривает.
- Импорт проверяет структуру и тип данных до записи в БД.
- Внешние/опасные ссылки из Markdown должны открываться через контролируемый системный механизм.
- Export без шифрования явно помечается как обычный backup.
- При добавлении password-protected backup это должно быть отдельной функцией, а не скрытым поведением.

14. Локализация и доступность

- RU / EN через translation files, без хардкода пользовательских строк.
- Переключение языка без перезапуска.
- Light / Dark / System.
- Контраст и читаемость ориентируются на WCAG AA.
- Keyboard navigation для основных сценариев.
- Фокус клавиатуры визуально различим.
- Ошибки не передаются только цветом.

15. Тестирование

Уровень	Что проверяем
Unit	Форматирование данных, domain logic, timer calculations, validation
Integration	Repositories + SQLite, import/export, task movement
E2E	Create project → task → focus → note → analytics
E2E	Onboarding → demo workspace
E2E	Theme/language switch
E2E	Export → reset → import → verify data
Manual	Tray, notifications, filesystem, packaging на платформах

15.1. Критические E2E-сценарии

- Создать проект → создать задачу → переместить в In Progress → завершить.
- Запустить focus на задаче → завершить сессию → увидеть её в analytics.
- Создать note → связать с task → закрыть приложение → открыть снова → проверить сохранение.
- Экспортировать данные → импортировать в чистую БД → сравнить ключевые сущности.
- Переключить RU/EN и theme → проверить отсутствие перезагрузки.

16. Структура проекта

```
kairo/
├── src/
│   ├── app/
│   ├── modules/
│   │   ├── dashboard/
│   │   ├── projects/
│   │   ├── tasks/
│   │   ├── notes/
│   │   ├── focus/
│   │   └── analytics/
│   ├── components/
│   ├── store/
│   ├── lib/
│   └── i18n/
├── src-tauri/
│   ├── src/
│   │   ├── commands/
│   │   ├── db/
│   │   ├── services/
│   │   └── main.rs
│   ├── migrations/
│   ├── tauri.conf.json
│   └── tests/
```

17. Roadmap

Этап	Содержание	Оценка
1. Foundation	Tauri, React, routing, DB, migrations, base design system	2-3 дня
2. Projects + Kanban	Projects, columns, tasks, DnD, filters	4-5 дней
3. Notes	Markdown, search, links	3-4 дня
4. Focus	Timer, task binding, tray, notifications	2-3 дня
5. Analytics	Queries, charts, dashboard	2-3 дня
6. Cross-cutting	Command palette, i18n, theme, export/import, onboarding	3-4 дня
7. Polish	UX states, animations, empty/error states, accessibility	2-3 дня
8. Packaging	Windows + second platform, installers	1-2 дня
9. Portfolio	Demo, screenshots, README, case study	1-2 дня

Итого ориентир: 20-29 рабочих дней. Для портфолио допустимо выпускать MVP раньше, если основные end-to-end сценарии уже отполированы.

18. MVP и приоритеты

Приоритет	Функции
P0 — обязательно	Projects, Kanban tasks, Notes, Focus timer, SQLite, Dashboard, command palette, theme, export/import
P1 — сильно желательно	Analytics, tray, notifications, demo data, RU/EN, Files
P2 — после MVP	Расширенные фильтры, дополнительные виды аналитики, более сложная кастомизация workspace

Главное правило: не добавлять функции, которые не усиливают основной цикл Project → Task → Focus → Note → Review. Лучше 6 глубоко проработанных возможностей, чем 20 поверхностных.

19. Definition of Done

- Можно создать проект и полностью пройти его workflow от задачи до аналитики.
- Kanban работает с drag & drop и сохранением порядка.
- Notes сохраняются между перезапусками.
- Focus session привязывается к task и попадает в историю.
- Dashboard показывает реальные данные из SQLite.
- Export → Import восстанавливает данные.
- Light/Dark/System работает.
- RU/EN работает без перезапуска.
- Приложение работает offline.
- Tray и notifications протестированы на целевой desktop-платформе.
- Нет постоянных console errors, очевидных memory leaks и зависаний в длительном сценарии.
- Есть production-like installer хотя бы для Windows и ещё одной платформы.
- README объясняет архитектуру, запуск и ключевые решения.

20. Что показывать в портфолио

20.1. Главный демо-сценарий на 30-60 секунд

- Открыть Kairo → Dashboard.
- Создать/открыть проект.
- Создать задачу через keyboard shortcut.
- Перетащить задачу по Kanban.
- Открыть note и связать её с задачей.
- Запустить Focus.
- Показать tray/notification.
- Открыть Analytics и показать, как сессия попала в статистику.
- Показать export/import или Settings как финальный desktop feature.

20.2. Что должно быть видно работодателю/клиенту

- Не только красивый UI, но и реальная локальная БД.
- Сложное состояние и связи сущностей.
- Drag & drop.
- Native desktop APIs.
- Таймер и фоновые сценарии.
- Charts и вычисляемая аналитика.
- Межплатформенная сборка.
- Тесты и продуманная архитектура.
- Внимание к empty/loading/error states.

20.3. Материалы

- 30–60 секунд screen-recording.
- 5–8 сильных screenshots.
- Public README.
- Установщик приложения.
- Короткий case study: проблема → исследование → решения → архитектура → результат.
- Таблица фактических performance/build metrics.

21. Итоговая продуктовая формула

Kairo = Project workspace + Kanban + Markdown notes + Focus timer + Analytics + Native desktop UX.

Проект должен восприниматься как законченный desktop-продукт, а не как набор независимых pet-project модулей. Каждая крупная функция должна быть связана с общей моделью проекта и возвращать пользователя в основной рабочий цикл.

Приоритет разработки: сначала надёжный data model и core workflow, затем native features, затем analytics и polish. В portfolio presentation акцент следует делать на инженерных решениях и цельности продукта, а не только на визуальной оболочке.

22. Примечание по происхождению решений

Это ТЗ является синтезом двух исходных материалов: Flowdesk и предоставленного FocusHub. Термины, функции и оценки, которые прямо присутствовали в исходных материалах, сохранены или объединены. Архитектурное решение сделать Project отдельной верхнеуровневой сущностью, связать Board с Project, а также расширить модель связями Note/Task/File — это рекомендуемая синтезация для цельности итогового продукта.

Некоторые числовые требования (например, cold start и размер сборки) оставлены как цели/ориентиры и должны быть подтверждены измерениями после реализации.